BRAUDE
College of Engineering, Karmiel

Software Engineering Department

Braude College

PLARA - Capstone Project Phase B
Production Line Augmented Reality Application

By:
Shlomi Fridman
Shahar Berenson

Advisor:
Dr. Naomi Unkelos Shpigel

Project Code:
25-2-D-9

Github

Google Drive

Table of Contents

Common Definitions and Acronyms

Term	Description
AR	Augmented Reality
DT	Digital Twin
IIoT	Industrial Internet of Things
PL	Production Line
QR-CODE	Quick-Response Code

Abstract

Understanding and monitoring production line operations remains a challenge for engineering students and educational staff alike, due to the complexity and abstraction of traditional tools. To address this, we designed a mobile Augmented Reality (AR) application that visualizes a real production line using Unity, integrating real-time data from QR-Code installed on the line, and includes in-app guides and a quiz to support learning. The system enables users to interact with digital twins of physical stations, track cart movement, and receive visual feedback, thereby enhancing both educational engagement and operational monitoring.

Keywords
Production Line Training, AR Instructional Guide, Engineering Education, Digital Twin (DT), Industrial Internet of Things (IIoT)

1. Introduction

Modern industrial and educational environments face significant challenges in accurately visualizing complex production line operations. Traditional monitoring systems often rely on static or abstract dashboards, which can limit the comprehension of students and trainees who are unfamiliar with underlying system processes. This gap in real-time feedback is a critical hurdle in educational settings, where hands-on understanding is essential for mastering system behavior. [1] [2]

To address this, the project proposes a mobile Augmented Reality (AR) application integrated with the Industrial Internet of Things (IIoT) thus offering improved user interaction and decision making [3] [4]. Built using the Unity engine, the application is specifically designed to serve as an interactive guide for new students. By providing a dynamic 3D visualization of a functioning production line and processing real-time data from QR-code sensors, the system allows users to track cart movements and station statuses through an immersive interface, and read guides on the production line.

The project aims to not only enhance production line transparency and provide monitoring services but also bridge the gap between industrial operations and educational training through an accessible, immersive AR experience.

2. Background

The foundation of the project rests on combining three key technologies:

- **Augmented Reality (AR) for Monitoring:** AR is presented as a powerful tool for industrial oversight, enabling the overlay of real-time sensor data onto physical machinery to assist in fault detection, predictive maintenance, and remote collaboration. [5] [6]
- **Industrial Internet of Things (IIoT):** IIoT acts as a driver for operational efficiency and automation, connecting physical systems and big data analytics to increase manufacturing productivity [1]. And incorporating real engineering challenges and remote laboratories into educational curricula can enhance student engagement and practical skills. [7]
- **Digital Twin (DT) Technology:** The synergy of AR and DT allows for the creation of virtual replicas that mirror physical objects in real time. This hybrid provides a deeper understanding of complex systems by allowing users to view with live data superimposed onto the physical environment around them. [8]

The practical application of these concepts is centered on the production line located at Lab D106, which simulates a modern manufacturing lifecycle (figure 1). The line includes stations such as:

- **Storage:** Managed by an Automated Storage Retrieval System (ASRS 2).
- **Creation:** Utilizing a BenchMill 6000 CNC machine and SCORBOT-ER V Plus robotic arm.
- **Review:** Using a Yaskawa Motoman-GP8 arm for quality assessment.
- **Assembly:** Operated by a SCORA ER-14 for precise component insertion.

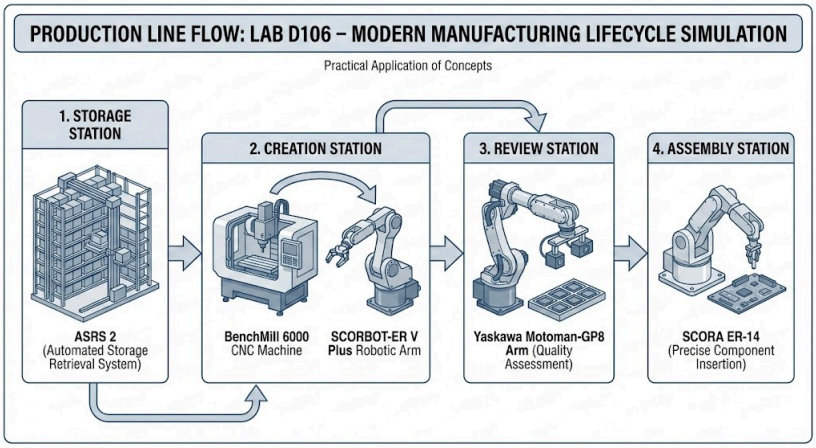


Figure 1: The manufacturing lifecycle of the production line in Lab D106

3. PLARA

3.1 Description

PLARA serves as the primary digital management and education system for the production line in Lab D106. By bridging the gap between theory and practice, the platform provides real-time monitoring of the manufacturing lifecycle (the architecture of which is illustrated in Figure 2) using a network of QR-Scanners to track the carts as they move through the line. Beyond mere monitoring (a flow of which is depicted in Figure 3), PLARA also functions as an interactive educational tool, it offers comprehensive guides for every station located on the production line, complemented by quiz and badges the students can earn, designed to reinforce industrial concepts and ensure a deeper, more engaging educational experience for students.

3.2 The PLARA System

3.2.1 Architecture Diagram

The system employs a client–server architecture (as depicted in Figure 2) that connects a physical production line with a mobile Augmented Reality (AR) application. An Android-based AR app developed in Unity communicates with a cloud-hosted Node.js and Express backend via HTTP and WebSocket protocols to support real-time interaction and data exchange. The backend, deployed on the Render platform, interfaces with an M5Stack controller on the production line using MQTT, receiving events from QR-code sensors that detect cart entry, exit, and movement between stations, ensuring real-time synchronization between the physical production line and its digital twin for monitoring and educational purposes. All user data is stored in a MongoDB Atlas database.

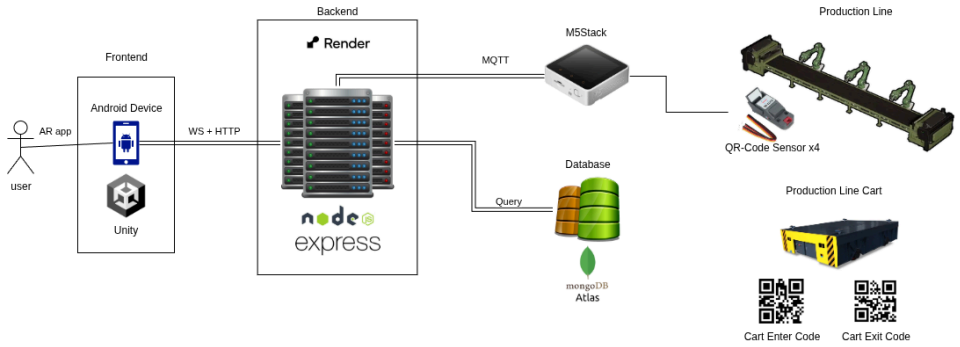


Figure 2: PLARA architecture diagram

3.2.2 Activity Diagram: New Station Occupancy

The New Station Occupancy process (as shown in Figure 3) begins when a QR code is scanned by a production line sensor. The sensor evaluates whether the QR represents a change in station occupancy. If no change is detected, the process terminates without further action. When a new occupancy is identified, the sensor publishes an MQTT message containing the updated status to the server. The server receives the message, updates the system status accordingly, and broadcasts the new status to the application. Upon receiving the update, the AR application does a self-sync which triggers the movement of the previous station occupant towards the next waypoint on the PL, moves the new station occupant to the station waypoint, and displays a notification indicating that the cart (the new occupant) has arrived. The process concludes with the AR environment being updated to reflect the new station state

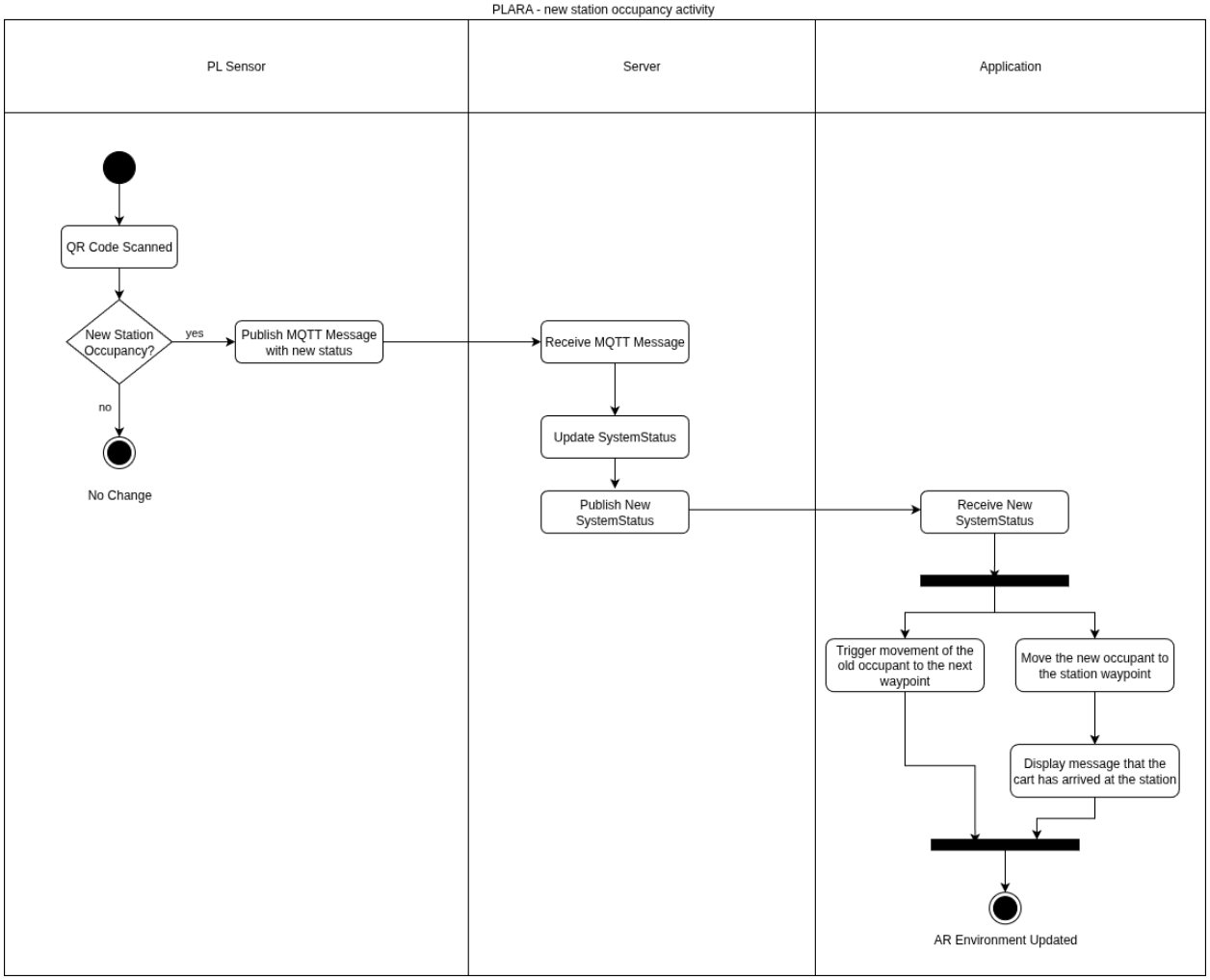


Figure 3: New station occupancy activity diagram

3.3 Development Timeline

3.3.1 Phase A

Phase A focused on conceptualization (the timeline of which is shown in Figure 4), early validation, and iterative prototyping of the AR-based production line visualization. It began in March 2025 with initial discussions with the advisor and the clients, during which system goals and constraints were defined. This phase progressed through a proof of concept demonstrating basic object movement, followed by successive prototypes that introduced waypoint-based motion and early AR representations of the production line. Throughout April to June 2025, the development was guided by recurring status, refinement, and client meetings, allowing continuous feedback to shape functionality and usability. Phase A concluded with an impressive prototype capable of representing cart movement and station interaction, establishing a solid foundation for full system development

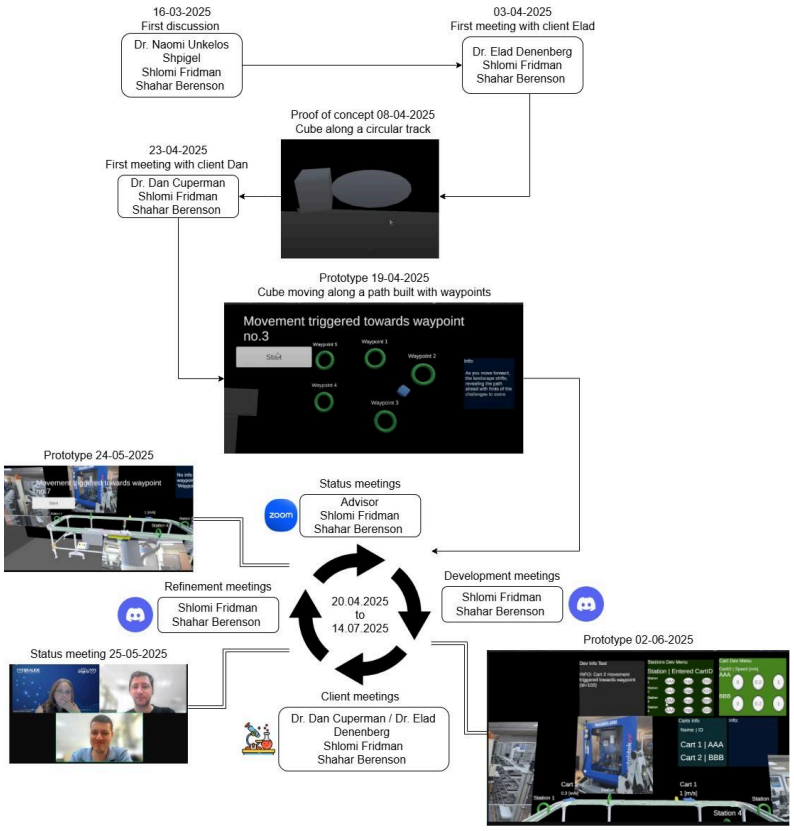


Figure 4: Phase A Development Timeline

3.3.2 Phase B

Phase B concentrated on system expansion, integration, and deployment, spanning from September 2025 through January 2026 (as depicted in Figure 5). This phase included the development and deployment of the server infrastructure, integration of the AR application with the backend server, and the implementation of additional user interface features such as menus and badges. Subsequent development iterations focused on integrating the application with the physical production line, culminating in full system integration by December 2025. The phase concluded with the first experimental evaluation conducted on the production line (see Sect. 3.7.1), followed by the presentation of results in the Phase B poster, marking the transition from development to evaluation and dissemination.

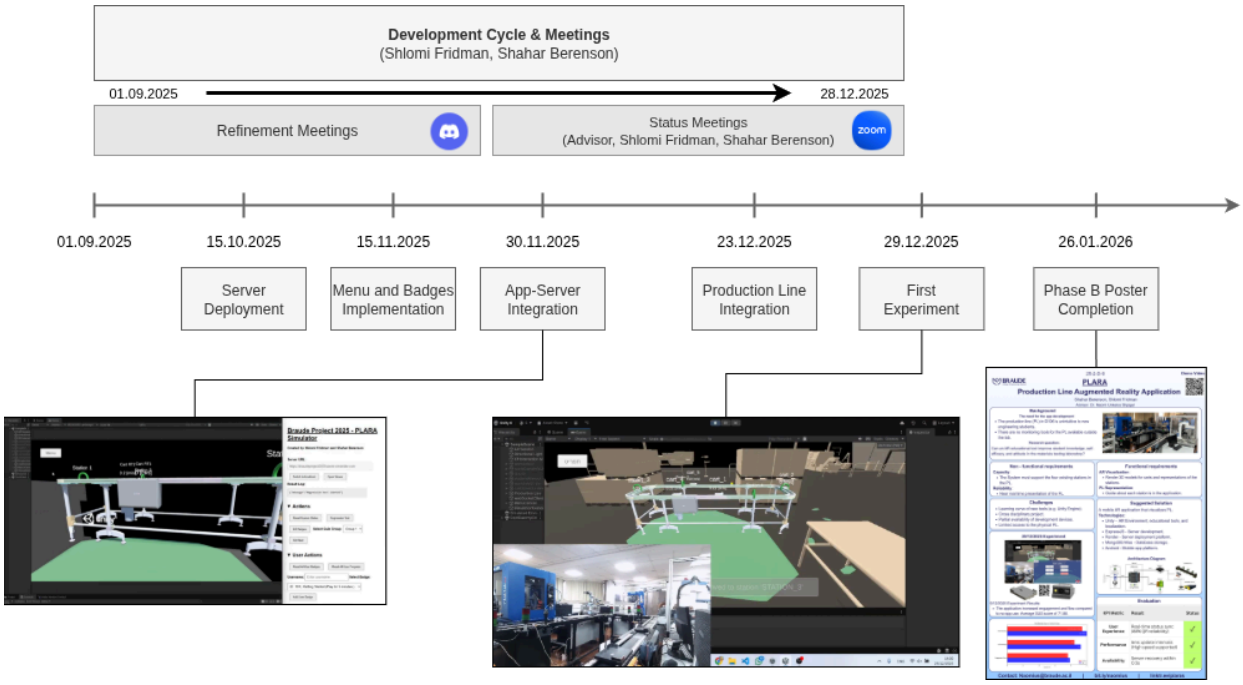


Figure 5: Phase B Development Timeline

3.4 Tools

3.4.1 Project

- **Draw.io** – Web-based diagramming tool for creating flowcharts, network diagrams, and system designs.
    - Used to create system and project diagrams.

- **Notepad++** – Open-source text and code editor for Windows.
    - Used to write and edit JSON configuration files.

- **Ivrit.ai** – An AI-based speech-to-text transcription service that supports Hebrew.
    - Used to transcribe interviews for research/documentation.

- **Blender** – Open-source 3D modeling and animation software.
    - Used to export and edit 3D models in .fbx format.

- **Discord** – A communication platform for text, voice, and video chat.
    - Used for internal team development meetings.

- **Zoom** – A video conferencing platform for remote communication and collaboration.
    - Used for meetings with the advisor and clients.

3.4.2 Application

- **Unity** – A cross-platform game and AR/VR engine used to build interactive 2D, 3D, and XR applications.
    - Main engine used to develop the client application.

- **Unity Cloud** – A suite of cloud services for Unity that includes version control, collaboration tools, and build automation.
    - Provides version control and collaboration for Unity projects.

- **Visual Studio** – An integrated development environment (IDE) by Microsoft for writing and debugging code in multiple languages, including C#.
    - Used to write and debug C# scripts for the Unity application.

3.4.3 Server

- **Visual Studio Code** – A lightweight, open-source code editor with support for debugging, Git integration, and extensions.
    - Used to develop and manage server-side code.

- **Postman** – A platform for testing APIs by simulating HTTP requests and inspecting responses.
    - Used to test and simulate HTTP requests/responses from the server.

- **Render** – A cloud platform for hosting web services and applications.
    - Platform used to deploy the server of the project.

- **GitHub** – A web-based platform for version control and collaborative software development using Git.
    - Version control system for managing the server codebase.

3.4.4 Sensors

- **UIFlow** – A visual programming environment designed for M5Stack microcontrollers.
    - Visual programming tool used to develop sensor scripts.

- **M5Burner** – An official utility for flashing firmware and configuring Wi-Fi settings on M5Stack devices.
    - Tool used to configure and flash WiFi settings onto the M5Stack controller.

3.5 Challenges

3.5.1 Learning Curve

During this project, we were first introduced to the concepts of augmented reality (AR) development and the manipulation of 3D objects within a simulated environment. Initially, we attempted to apply principles and techniques familiar from 2D development. However, as the project evolved and its requirements became clearer, we realized that a deeper understanding of new concepts was necessary.
These included manipulating 3D objects in Unity, maintaining their visibility within the AR camera's field of view, and integrating 2D elements where appropriate, such as the main menu and textual overlays displayed above stations and carts. After overcoming the initial learning curve and gaining a solid understanding of the core principles of Unity-based AR development, we became more comfortable with the tools and workflow, allowing the project to progress smoothly.

3.5.2 Cross Disciplinary Project

This project was a collaborative effort between the Software Engineering and Mechanical Engineering departments. The software engineering team acted as the developers, while the mechanical engineering lecturers served as the clients. Initially, this collaboration posed challenges, as the clients provided a high-level vision of the desired product, requiring us to translate it into well-defined requirements and core system concepts.
Throughout the project, we developed multiple proof-of-concept implementations to validate feasibility and to ensure that the proposed solutions could be completed within the given timeframe. Working with non-software engineering clients proved to be highly beneficial, as it required us to break down technical requirements into simpler, more accessible terms. This process helped expose potential design flaws, refine our assumptions, and improve the overall logical structure of the system. Overall, the collaboration was a highly enriching and valuable learning experience.

3.5.3 Limited Access to The Physical Production Line

Access to the physical production line, located in Lab D106, was limited, as the facility was not open to the public and required the presence of lab staff for each integration and testing session. This restriction introduced challenges, as system logic and design decisions had to be developed and validated during a small number of scheduled lab sessions.

As a result, certain ideas could not be fully realized. One such example was the concept of enabling carts to move at real-time speeds based on sensor-provided velocity data. Although the application was designed to support this functionality upon receiving the relevant data, it ultimately proved infeasible due to limitations in the available lab tools and infrastructure.

3.6 Lessons Learned

- Despite conducting thorough research and defining detailed requirements following interviews with the clients, we encountered limitations related to hardware availability and capabilities. Access to the sensors was obtained late in Phase A (timeline of which is depicted in Figure 4), and their practical testing and limitation analysis only began during Phase B (timeline is shown in Figure 5). As a result, the requirements related to handling real-time cart speed data could not be fully satisfied.

  While the system logic and software implementation were completed to support the processing of speed data, the available hardware was unable to provide the required data reliably. This experience highlighted the importance of validating hardware capabilities early in the development process before committing to full software implementation. Nevertheless, the system was designed with extensibility in mind, and support for speed data has been implemented to allow future development once suitable hardware becomes available.

- One of the initial system requirements specified support for only a single active cart on the production line. However, during the design and implementation of the cart-related logic in the AR environment, we identified an opportunity to abstract cart behavior. Each cart was designed to operate independently, with its logic based solely on its own state and movement, without reliance on the presence or behavior of other carts. After completing the implementation, we first validated the system using a simulation with a single active cart. Once satisfactory results were achieved, we expanded the simulation to include three concurrently active carts, all of which were represented accurately within the AR environment. Following successful laboratory integration with real-world sensors, the system was further tested with five active carts. These tests demonstrated a near real-time and faithful representation of cart behavior, meeting both our expectations and surpassing those of the clients.

  The key takeaway gained from this experience was the importance of maintaining an open mindset during system design and decomposing logic into its most fundamental components. This approach enabled scalability beyond the original requirements and led to results that exceeded initial expectations.

3.7 Evaluation

3.7.1 Experiment

On 29.12.2025, an experiment was conducted with 16 Mechanical Engineering students who had no prior exposure to the production line. The participants were divided into two groups:

1. Application Group: Participants used the developed application to observe both the physical production line and its virtual representation. They read the instructional guide on each station within the application and completed a quiz through the app.
2. Control Group: Participants observed only the physical production line. They accessed the instructional materials via Google Docs and completed the quiz using Google Forms.

After a 15-minute session, all participants completed a feedback questionnaire designed to assess engagement and understanding of the production line. In addition, the Application Group completed a System Usability Scale (SUS) questionnaire and provided feedback and suggestions regarding the PLARA system.

Results

- The Application Group received better scores in the PL quiz, and had a higher engagement score (although not by much) as depicted in Figure 6.
- The Application Group assigned the system an average SUS score of 71.88.

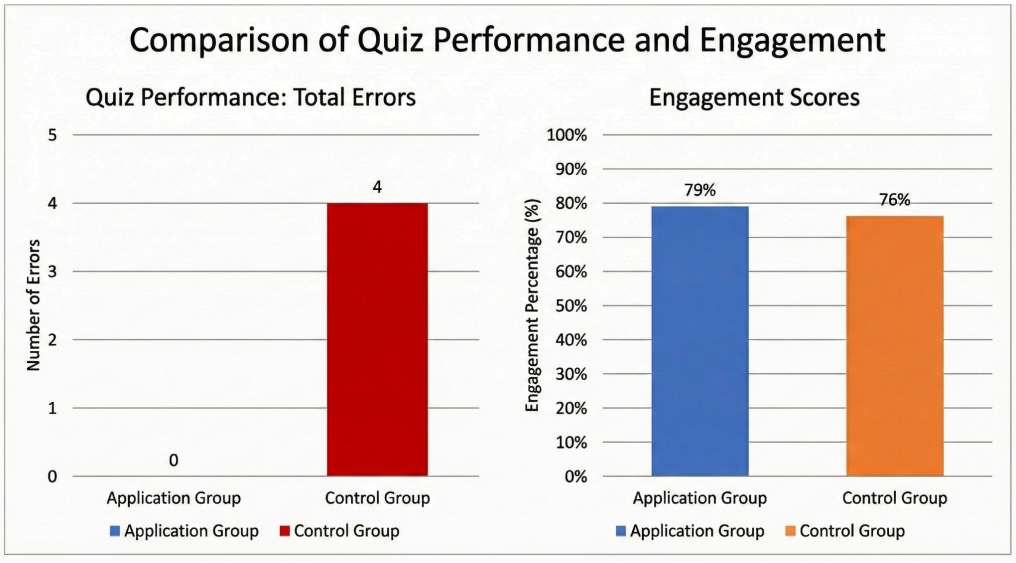


Figure 6: Comparison between the Application Group and the Control Group

Conclusions

Although there were not many participants. We can say, based on the experimental data from December 29, 2025, and the results illustrated in Figure 6, we can draw the following conclusions regarding the impact of the PLARA system on student learning and engagement:

1. **Superior Learning Outcomes**: The most significant finding is the disparity in quiz performance. The Application Group achieved a perfect record with **0 errors**, whereas the Control Group accumulated **4 errors**. This suggests that the integration of virtual representations and in-app instructional guides provides a more effective cognitive framework for understanding complex production lines than traditional methods like Google Docs.
2. **Enhanced Student Engagement**: The application successfully boosted learner interest, albeit by a competitive margin:
    - **Application Group:** 79% engagement.
    - **Control Group:** 76% engagement.
    - ★ While the 3% difference is modest, it indicates that the interactive nature of the application is at least as engaging, if not more so, than standard digital tools.
3. **System Usability (SUS):** The Application Group rated the system with an average SUS score of **71.88**. According to standard SUS benchmarks, a score above 68 is considered "Above Average" and falls within the "Good" range of usability. This confirms that the PLARA system is user-friendly and accessible for students with no prior exposure to the equipment.

The experiment demonstrates that the PLARA application is a highly effective educational tool for Mechanical Engineering instruction. By eliminating errors in the quiz and maintaining high engagement levels, the system proves to be a superior alternative to traditional observational learning. The "Above Average" usability score further validates the system's readiness for broader implementation in technical training environments.

3.7.2 KPI Metrics

We are pleased to report that the system successfully met all defined key performance indicator (KPI) metrics, with the exception of those related to cart speed measurement, as the speed sensor didn't meet demands.

Category	**Key Performance Indicator (KPI)**	**Target / Threshold**	**Result**
User Experience	Real-time Station Status Latency	**< 100ms** from event change	**< 100ms** from event change
Performance	Data Update Frequency	Every **10ms**	Every 5**ms**
Availability	OS Compatibility	**Android 15** or higher	**Android 7** or higher
Reliability	Session Stability	**0 crashes** per 30 mins of continuous use	**0 crashes** during more than one hour of continuous use.
Reliability	Connection Resilience	Auto-reconnect within **1 minute**	Auto-reconnect within **0.5s** of disconnection
Reliability	Data Integrity	**Zero data loss** during server disconnects	**Zero data loss** during server disconnects

4. References

[1] Peter, O., Pradhan, A., & Mbohwa, C. (2023). Industrial internet of things (IIoT): opportunities, challenges, and requirements in manufacturing businesses in emerging economies. Procedia Computer Science, 217, 856-865.

[2] Shabur, M. A. (2024). A comprehensive review on the impact of Industry 4.0 on the development of a sustainable environment. Discover Sustainability, 5(1), 97.

[3] Liberty, J. T., Sun, S., Kucha, C., Adedeji, A. A., Agidi, G., & Ngadi, M. O. (2024). Augmented reality for food quality assessment: Bridging the physical and digital worlds. *Journal of Food Engineering*, *367*, 111893.

[4] Bucsai, S., Kučera, E., Haffner, O., & Drahoš, P. (2020, January). Control and monitoring of IoT devices using mixed reality developed by unity engine. In 2020 Cybernetics & Informatics (K&I) (pp. 1-8). IEEE.

[5] Syed, T. A., Siddiqui, M. S., Abdullah, H. B., Jan, S., Namoun, A., Alzahrani, A., ... & Alkhodre, A. B. (2022). In-depth review of augmented reality: Tracking technologies, development tools, AR displays, collaborative AR, and security concerns. *Sensors*, *23*(1), 146.

[6] Pavanatto, L., North, C., Bowman, D. A., Badea, C., & Stoakley, R. (2021, March). Do we still need physical monitors? an evaluation of the usability of ar virtual monitors for productivity work. In *2021 IEEE virtual reality and 3D user interfaces (VR)* (pp. 759-767). IEEE.

[7] Diogo, R. A., dos Santos, N., & Loures, E. D. F. R. (2023). Improving Brazilian Engineering Education: real engineering challenges in an IIoT undergraduate course. In Designing Smart Manufacturing Systems (pp. 35-57). Academic Press.

[8] Yin, Y., Zheng, P., Li, C., & Wang, L. (2023). A state-of-the-art survey on Augmented Reality-assisted Digital Twin for futuristic human-centric industry transformation. Robotics and Computer-Integrated Manufacturing, 81, 102515.

5. Appendixes

5.1 Links

- Github Repository
- Google Drive documents
- Phase A Book
- Demonstration Video
- Feedback linktree

5.2 User Guide

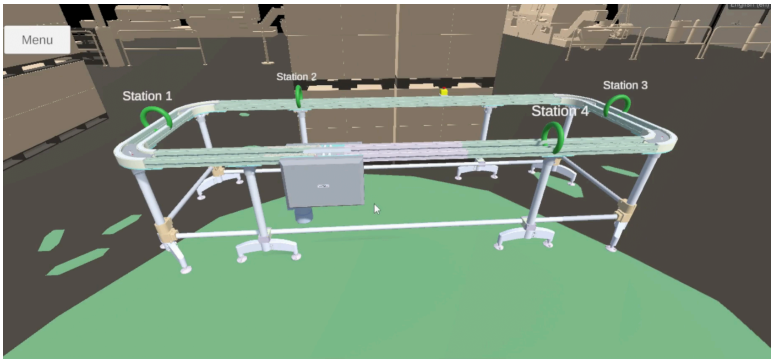
This guide provides instructions for using the PLARA application. its normal operational workflow for users and information on how to navigate the system successfully.

Make sure to download PLARA before use.

5.2.1 Getting Started

1. **Launch the App**
   Open the application on your Android device.
2. **Login**
   2.1. Enter your **username**.
   2.2. Choose a **group** from the dropdown menu (**Group 1** or **Group 2**).
   2.3. Tap the flag to switch localization between English and Hebrew.
   2.4. Tap **Enter** to access the main application.

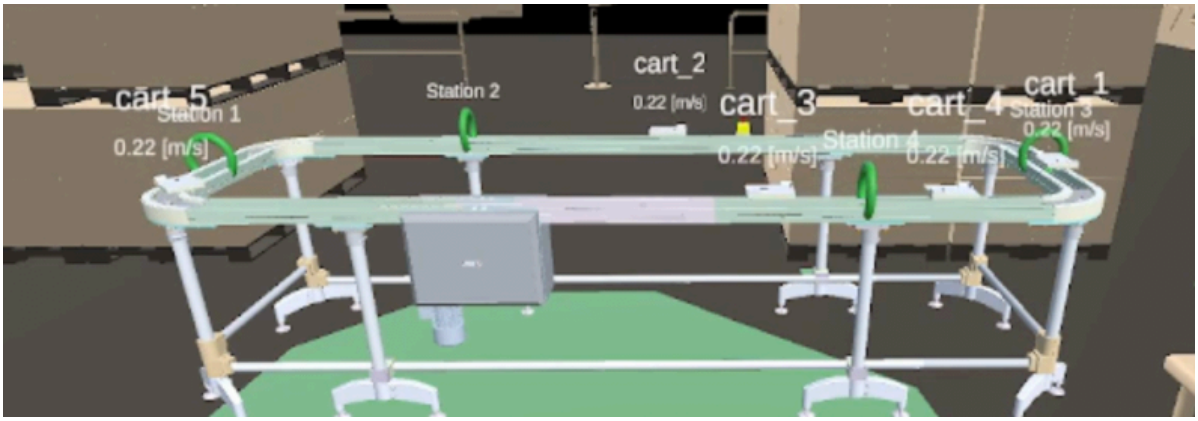
3. **Place the Production Line**
   3.1. After entering, tap on the green floor to **place the production line**.



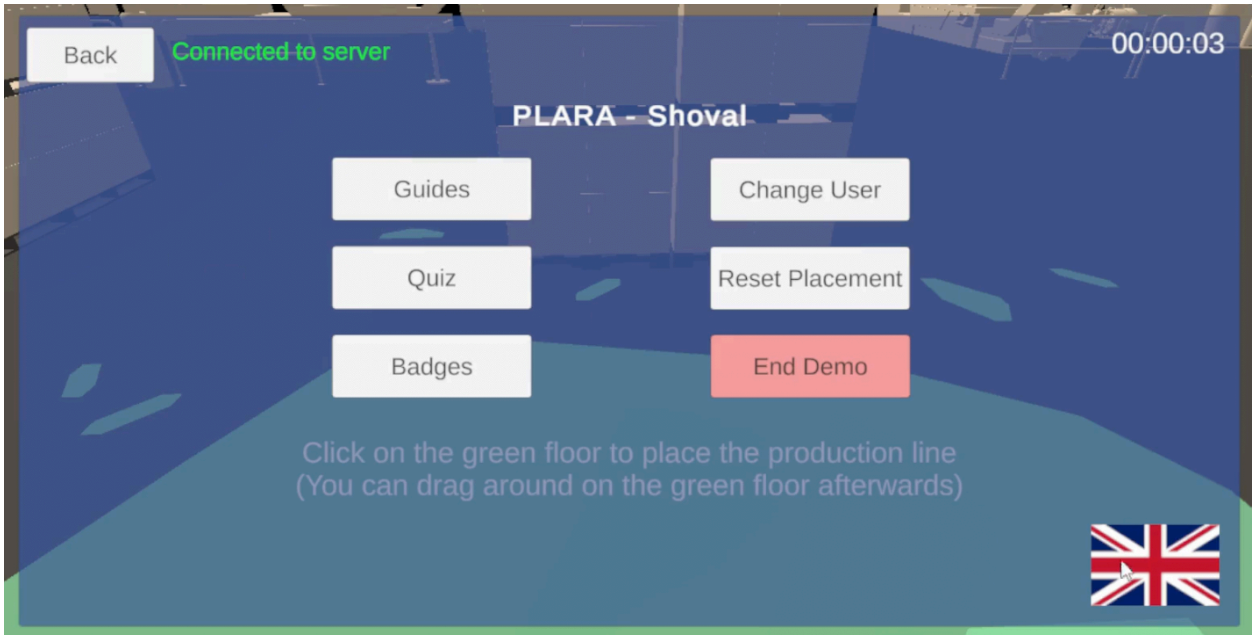
5.2.2 Main Interface Overview

Once the production line is placed, you will see the **carts moving along the line**. The interface allows you to:

- Observe the movement of the carts when the production line and the sensors are active.



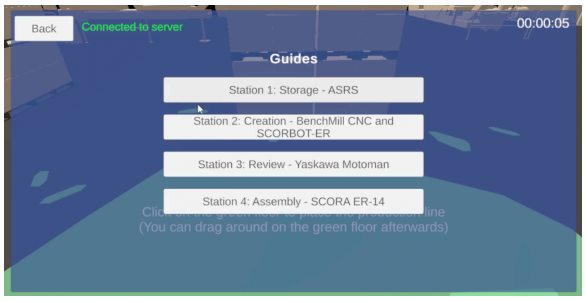
- Open the menu at top left to access additional options.

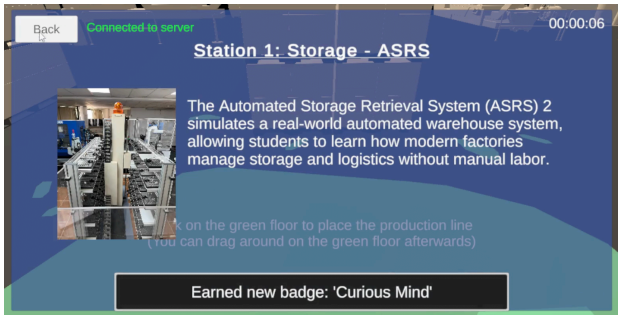


5.2.3 Top Menu Options

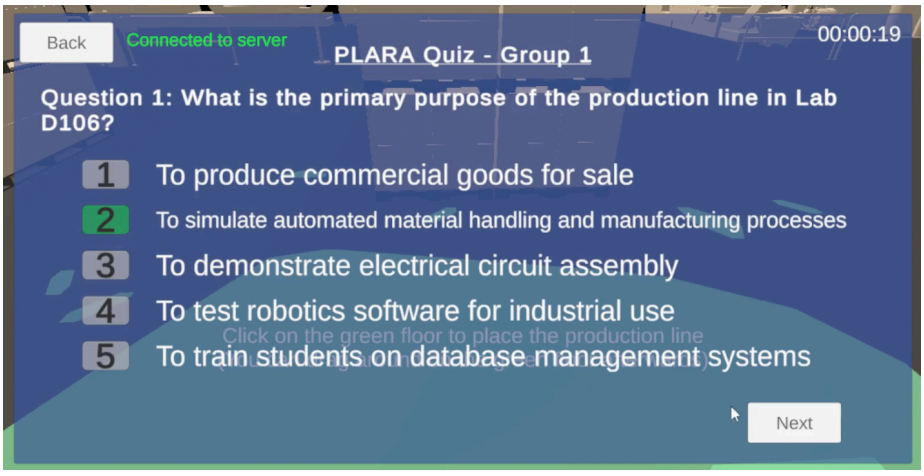
Tap the **menu button** at the top left section to access the following features:

1. **Guides**
    - Tap on each guide button to view their explanations and images for that station's guide.

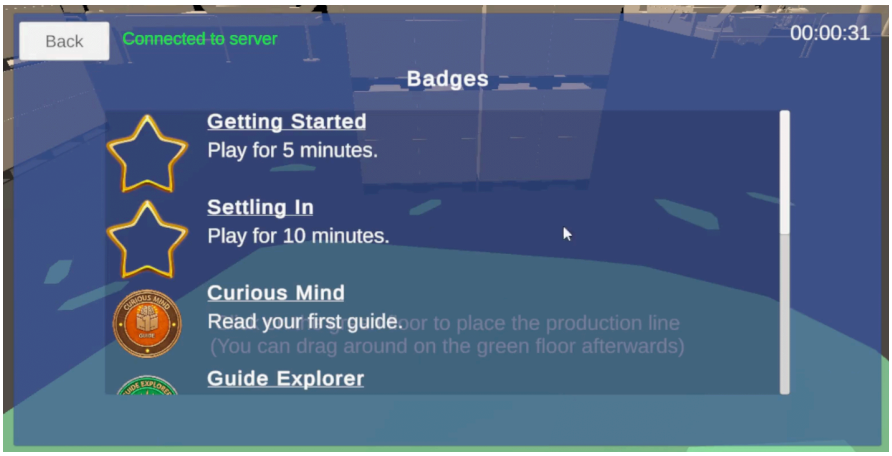




2. **Quiz**
    - To complete the 5-question quiz, select an answer by tapping the answer number in the question.
    - Tap Next or Previous to navigate between answered questions and return to the current one.



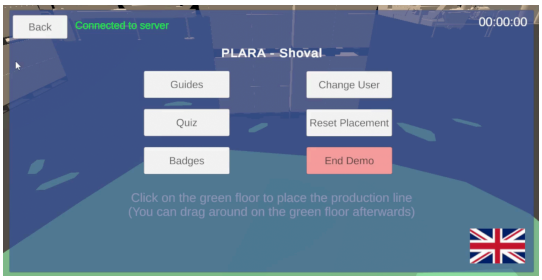
3. **Badges**
    - View **badges** related to activities performed in the app.



4. **Reset Placement**
    - Place the production line to a different position as needed.
5. **Change User**
    - Sign out from your current user account (See Sect. 2).
6. **End Demo**
    - Displays your last completed quiz results.
    - QR code & button for external links:
        - Demonstration video
        - Station operating videos
        - Consent Form for Online Research
        - PLARA APKs.
        - Feedback form.
        - Station guides that are the same as in the app.
        - Google form quiz that is the same as in the app.
    - Tap Reset Demo to log out (See Sect. 2). Or close the app.

7. Tap the flag to switch localization between English and Hebrew.

- Tap on the “Back” button to return to the main menu from any screen.

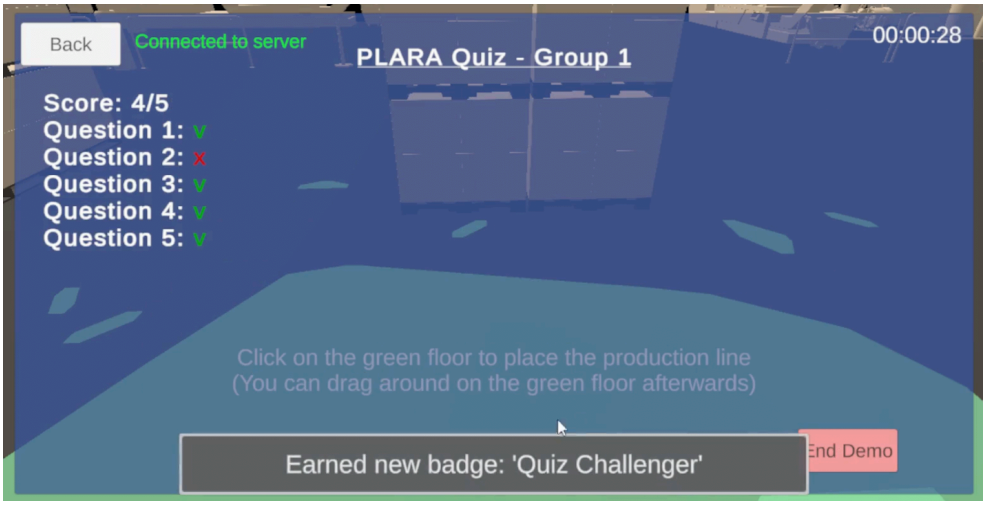


5.2.4 Using the Production Line

1. Observe the movement of the carts when the production line is active.
2. Use the **“Reset Placement”** option if you wish to change the position of the line.
3. Monitor the interaction between carts and stations during the simulation.

5.2.5 Completing Quiz

- Select Quiz from the menu.
- Answer all 5 questions.
- Tap End Demo if you desire to end the session.



5.2.6 Ending Demo

- In the menu or post quiz, tap End Demo to finish the current session, view your last quiz and access external links.
- Feedback and demo evaluation to be submitted via the Links & QR provided, they are available in English or Hebrew.

5.3 Maintenance Guide

This guide is intended to support ongoing use, updates, and improvements of the PLARA system after project completion. It provides instructions for system setup, maintenance, and deployment of updates.

5.3.1 System Environment

1. **Software Requirements**
    - Unity with ARCore support for Android.
    - Node.js and Express.js for the backend server
    - Python 3.x for controller scripts in Lab D106
    - PLARA source code repository (GitHub or Unity Cloud)
2. **Hardware Requirements**
    - Android device (Min ver. 7 API 24) with ARCore support.
    - Development PC with at least 8GB RAM and 50GB free disk space
    - QR code scanner units for triggering events, M5Stack Core2 controller, extension cables, and a Pa.Hub splitter in Lab D106.
3. **Server Environment**
    - Express.js server deployed locally or remotely (e.g. Render).
    - MongoDB deployed locally or remotely (e.g. MongoDB Atlas).

5.3.2 Installation Guide

1. **Unity Setup:**
   - 1.1. Clone the applications scripts: https://github.com/ShlomiFridman/BraudeProject2025App
   - 1.2. **Open Unity Project**
     - 1.2.1. Launch Unity → Open Project → Select PLARA folder.
   - 1.3. **Set ARCore and Android Build Settings**
     - 1.3.1. Ensure ARCore support is enabled.
     - 1.3.2. Set platform to Android in Build Settings.
     - 1.3.3. Configure localization settings if needed.
   - 1.4. **Update environment variables**
     - 1.4.1. Update environment variables in .env file, like API keys, database credentials, or app settings.
   - 1.5. **Build APK**
     - 1.5.1. `File → Build Settings → Build`
     - 1.5.2. Install APK on Android device for testing.

2. **Server Setup**
   - 2.1. Clone the server repository: https://github.com/shachar700/PLARAServer.git
   - 2.2. Ensure NodeJS is installed.
   - 2.3. Install dependencies: `npm install`.
   - 2.4. Start server: `npm run dev`.
   - 2.5. Open in browser: http://localhost:5000
   - 2.6. Confirm server is running and accessible.
   - 2.7. Init the badges in the database (if needed) by clicking on the “Init Badges” button in the server sim page (see Sect. 5.3.5).

3. **Python Controller Setup (Lab D106)**
   - 3.1. Ensure Python 3.x is installed.
   - 3.2. Navigate in the server repository to `/src/assets/sensor_codes/`
   - 3.3. Copy content of `QR_SensorCode.py`.
   - 3.4. Place the copied python script in the controller system: `https://flow.m5stack.com/`
   - 3.5. Test scripts to confirm QR triggers are functioning correctly.

5.3.3 Updating the System

1. **Unity App**
    1.1. Update the server's urls in `/Assets/Scripts/ProjectClasses/AppConstants.cs`.
        1.1.1. For local deployment: update the ServerLocalAddress field.
        1.1.2. For remote deployment: update the ServerDeploymentAddress field.
        1.1.3. The app will connect to local/remote based on the IsLocalServer flag.
    1.2. Update the guides json file in /Assets/Resources/jsonFiles/Guides.json
2. **Server**
    2.1. Update the `/.env file`.
        2.1.1. To update the db uri, update the MONGODB_URI field.
        2.1.2. To update the MQTT uri, update the MQTT_BROKER_URI field.
    2.2. Update Group X's quiz in src/assets/files/QuizGroup{x}.json.
3. **Python Controller:** Modify scripts for new triggers, monitor feedback or lab logic.

5.3.4 Deployment

- Build a new APK for Android devices.
- Update server code on cloud service (e.g. Render).
- Ensure QR sensors and controller scripts are operational.
- Confirm app-server-controller interaction is functioning.

5.3.5 Server Simulator

- In the server sim (http://localhost:5000/sim): you can trigger events, simulate cart movement speed, station occupancy and update records in the database.

★ Used for development and testing to validate system behavior without relying on live hardware.

5.3.6 Maintenance Tips

- Always work on a different branch than Master, and create Pull Requests only after testing..
- Keep Unity, Node.js, and Python updated.
    - Update the app and server source code if necessary.
- Document any changes made, and keep the changelog up to date.
- Test system integration after every major change via the simulator.
- Maintain hardware (QR sensors, controller) in Lab D106 for reliable operation.

5.3.7 Future Improvements

- Anchor the production line presentation to the physical object.
- Add speed velocity of the carts from sensors.
- Add feedback to the controller's monitor.
- Add logic to alert the user when a cart has not reached a station.
- In Hebrew localization, add support to Right-to-Left menus and screens.
- Add Hebrew localization where missing: dropdown, pop-ups, and production line hovered text.
- Add connected users list to the server dashboard.
- Improve functionality and visibility of the dashboard.
- Add video player of stations operating to Guides.
- Add particle effects.
- Add sound effects.
- Add support to iOS.
- Add support to WebGL.
- Add a home screen.
- Add credits screen.
- Make a web landing page.
- Add support to new stations.
- Add lecturer control in the dashboard over the details displayed in the app about guides and quiz.
- Add app check for updates routine so that users don't need to manually download newer builds.
- Add the app to the Vuforia/Google Play/Apple app store.